Truck Booking & Tracking System

Project Documentation

Submitted by

Yaksh Bhesaniya [25M0313]

Aditya Awasthi [25M0307]

Bahul Mishra [25M0317]

Submitted To

Prof. Surya Durbha

Objective

To develop a GIS-enabled web application that allows users to **hire nearby heavy vehicles** (trucks and other freight vehicles) and **track their movement** from pickup to destination in real time.

The system simulates truck movement on a map, manages bookings via REST APIs (user and driver panels), stores vehicle and booking data in a JSON file, and lays the foundation for future **cost and profit optimization** for both users and vehicle owners.

Documented Algorithm with Flowchart

Steps in the Algorithm

1. **Initialize Project & Load Configuration**
    - Start the Flask backend and load configuration (port, data file paths).
    - Read initial data from backend/database/data.json including vehicles, locations, and existing bookings.
    - Start background scheduler / simulator support modules.
2. **User Booking Workflow**
    - User opens the **User Panel** (frontend) and views the map and list of predefined locations.
    - User selects **pickup** and **drop** locations and enters cargo details if required.
    - Frontend sends a POST request to the User API (/api/user/book) with pickup and drop coordinates.
3. **Booking Creation & Vehicle Allocation**
    - Backend validates the request (locations present, coordinates valid).
    - System searches the list of trucks in data.json to find the **nearest available vehicle** based on geographic distance from the pickup point.
    - A new **Booking** object is created with status PENDING or ASSIGNED, and the selected truck is linked to this booking.
    - Booking entry is stored in memory and persisted back to data.json.
4. **Driver Notification & Acceptance**

- The **Driver Panel** fetches bookings via Driver API endpoints (/api/driver/bookings).
- Driver views the list of active bookings and clicks **Accept** for one booking.
- Backend updates booking status to ACCEPTED, updates truck status to ON_TRIP, and triggers the **simulation** for that booking.

5. **Trip Simulation – Phase 1: Move to Pickup**
    - The **Simulator Service** starts a background loop for the selected booking.
    - It interpolates coordinates from the truck's current position to the pickup location and updates the truck position every fixed time step (e.g., 0.8 seconds).
    - At the end of this phase, booking status becomes ARRIVED_DRIVER.
    - After a small wait (or explicit driver action like "Mark Loaded"), booking status changes to LOADED.
6. **Trip Simulation – Phase 2: Move to Drop**
    - Simulator computes a path from **pickup** to **drop** (in the prototype, a straight-line interpolation between coordinates).
    - The truck's coordinates are updated periodically towards the drop location.
    - During this phase, the map in the Tracking View and Driver/User Panels continuously polls the backend to fetch updated coordinates and statuses.
7. **Completion & Cleanup**
    - When the drop location is reached, simulator sets booking status to COMPLETED.
    - Truck status is reverted back to AVAILABLE.
    - All updates are written to data.json.
    - The system is ready to accept the next booking.

Textual Flowchart

(Use this as your **flowchart content** in the report/PPT – similar style to the emergency sample.)

START / Initialize

|

v

Load Configuration & Data

– Start Flask server

– Load vehicles, locations, bookings from data.json

|

v

User Creates Booking

– User selects pickup & drop on map

– Send POST /api/user/book with coordinates

|

v

Validate & Find Nearest Truck

– Check request fields

– Search available trucks

– Compute distance to pickup

– Choose nearest available truck

|

v

Create Booking & Assign Truck

– Create Booking object with status PENDING/ASSIGNED

– Persist to data.json

|

v

Driver Views & Accepts Booking

– Driver Panel calls /api/driver/bookings

– Driver clicks **Accept**

– Booking status → ACCEPTED

– Truck status → ON_TRIP

|

v

Start Simulation Thread

– Simulator starts for this booking

|
v

Phase 1: Move to Pickup

– Interpolate coordinates
– Update truck position every Δt
– On arrival: status ARRIVED_DRIVER
– Wait / Mark Loaded → status LOADED

|
v

Phase 2: Move to Drop

– Interpolate coordinates to drop
– Periodically update position & status
– Frontend map polls for live updates

|
v

Trip Completed

– Status COMPLETED
– Truck status back to AVAILABLE
– Persist final state to data.json

|
v

END

Pseudo-code

The pseudo-code is written in the same descriptive style as the emergency navigator sample: START, FUNCTION, uppercase keywords, etc.

High-Level Control Flow

START

FUNCTION main():

LOAD configuration (port, data_file_path)

LOAD data from JSON (vehicles, locations, existing bookings)

INITIALIZE Flask app and API routes

START background scheduler for simulation support

```
  RUN Flask server
END
```

Booking Creation & Matching

```
FUNCTION create_booking(pickup_location, drop_location, cargo_details):
  VALIDATE pickup_location and drop_location
  nearest_truck = find_nearest_available_truck(pickup_location)

  IF nearest_truck == NONE:
    RETURN error "No trucks available"
  ENDIF

  booking_id = GENERATE_UNIQUE_ID()
  booking = NEW Booking(
    id = booking_id,
    pickup = pickup_location,
    drop = drop_location,
    truck_id = nearest_truck.id,
    cargo = cargo_details,
    status = "ASSIGNED"
  )

  UPDATE nearest_truck.status = "ON_TRIP"

  SAVE booking and truck to data store (JSON)
  RETURN booking
END
FUNCTION find_nearest_available_truck(pickup_location):
```

```
  best_truck = NONE
  best_distance = INFINITY

  FOR each truck IN trucks_list:
    IF truck.status == "AVAILABLE":
      d = distance(pickup_location, truck.current_location)
      IF d < best_distance:
        best_distance = d
        best_truck = truck
      ENDIF
    ENDIF
  ENDFOR

  RETURN best_truck
END
```

Driver Acceptance Flow

```
FUNCTION driver_accept_booking(booking_id, driver_id):
  booking = GET booking BY id
  IF booking == NONE:
    RETURN error "Booking not found"
  ENDIF

  truck = GET truck BY booking.truck_id
  IF truck == NONE:
    RETURN error "Truck not found"
  ENDIF
```

```
    booking.status = "ACCEPTED"
    booking.driver_id = driver_id

    SAVE booking to data store

    START simulate_booking(booking.id) IN background_thread

    RETURN booking
END
```

Simulation Logic

This captures the core of backend/services/simulator.py.

```
FUNCTION simulate_booking(booking_id):
    booking = GET booking BY id
    truck = GET truck BY booking.truck_id

    IF booking == NONE OR truck == NONE:
        RETURN
    ENDIF

    '------ Phase 1: Move truck to pickup location ------'
    path_to_pickup = generate_route(truck.current_location, booking.pickup)

    FOR each point IN path_to_pickup:
        truck.current_location = point
        SAVE truck to data store
        SLEEP(0.8 seconds)
```

```
  ENDFOR

  booking.status = "ARRIVED_DRIVER"
  SAVE booking

  'Option A: automatic loading with delay'
  SLEEP(3 seconds)
  booking.status = "LOADED"
  SAVE booking

  '------ Phase 2: Move truck to drop location ------'
  path_to_drop = generate_route(booking.pickup, booking.drop)

  FOR each point IN path_to_drop:
    truck.current_location = point
    SAVE truck to data store
    SLEEP(0.8 seconds)
  ENDFOR

  booking.status = "COMPLETED"
  truck.status = "AVAILABLE"

  SAVE booking and truck
END
```

Routing & Cost Estimation (if cost_calculator/route_optimizer used)

```
FUNCTION generate_route(start_point, end_point):
  'Prototype: simple straight-line interpolation'
```

```
    steps = 20
    route = EMPTY_LIST

    FOR i FROM 0 TO steps:
        t = i / steps
        lat = start_point.lat * (1 - t) + end_point.lat * t
        lon = start_point.lon * (1 - t) + end_point.lon * t
        APPEND (lat, lon) TO route
    ENDFOR

    RETURN route
END
FUNCTION estimate_trip_cost(distance_km, base_rate_per_km, cargo_weight):
    variable_cost = distance_km * base_rate_per_km
    weight_factor = 1.0

    IF cargo_weight > THRESHOLD:
        weight_factor = 1.2
    ENDIF

    total_cost = variable_cost * weight_factor
    RETURN total_cost
END
```

API Handler Pseudo-code

```
ROUTE POST /api/user/book:
    READ pickup_location, drop_location, cargo_details FROM request
    booking = create_booking(pickup_location, drop_location, cargo_details)
```

RETURN booking AS JSON

ROUTE GET /api/driver/bookings:

bookings = GET all bookings WHERE status IN ("ASSIGNED", "ACCEPTED", "ARRIVED_DRIVER", "LOADED")

RETURN bookings AS JSON

ROUTE POST /api/driver/accept/<booking_id>:

driver_id = GET from request or session

booking = driver_accept_booking(booking_id, driver_id)

RETURN booking AS JSON

ROUTE GET /api/track/<booking_id>:

booking = GET booking BY id

truck = GET truck BY booking.truck_id

RETURN booking + truck.current_location AS JSON

Data Structures

Based on backend/models/booking.py, backend/models/vehicle.py, and the JSON layout used by the app.

Vehicles (Dynamic Data)

Represents all heavy vehicles tracked by the system.

Vehicle:

id: STRING 'Unique truck ID'

name: STRING 'Truck or fleet name'

current_location:

lat: FLOAT

lon: FLOAT

status: STRING 'AVAILABLE | ON_TRIP | UNAVAILABLE'

capacity_kg: FLOAT 'Cargo capacity (optional)'

base_rate_per_km: FLOAT 'Base fare rate'

Example JSON entry:


```
{
 "id": "TRUCK-001",
 "name": "IITB-Logistics-1",
 "current_location": { "lat": 19.1334, "lon": 72.9133 },
 "status": "AVAILABLE",
 "capacity_kg": 8000,
 "base_rate_per_km": 20.0
}
```


Bookings (Dynamic Data)

Each user request mapped to a specific truck and route.

Booking:

id: STRING 'Unique booking ID'

user_name: STRING (optional)

pickup:

lat: FLOAT

lon: FLOAT

label: STRING

drop:

lat: FLOAT

lon: FLOAT

label: STRING

truck_id: STRING 'Assigned Vehicle.id'

driver_id: STRING (optional)

cargo_weight_kg: FLOAT (optional)

distance_km: FLOAT (estimated)

estimated_cost: FLOAT

status: STRING 'PENDING | ASSIGNED | ACCEPTED | ARRIVED_DRIVER | LOADED | COMPLETED'

created_at: DATETIME

Example:


```
{
 "id": "BOOK-123",
 "pickup": { "lat": 19.1310, "lon": 72.9140, "label": "Main Gate" },
 "drop": { "lat": 19.1180, "lon": 72.9090, "label": "Warehouse" },
 "truck_id": "TRUCK-001",
 "distance_km": 7.5,
 "estimated_cost": 150.0,
 "status": "ASSIGNED",
 "created_at": "2025-11-23T10:10:00Z"
}
```


Locations (Static / Semi-Static Data)

Predefined campus / city locations for easier user selection.

Location:

name: STRING

lat: FLOAT

lon: FLOAT

Example:


```
{
 "name": "IIT Bombay Main Gate",
 "lat": 19.1334,
 "lon": 72.9133
```

}

Stored in a list inside data.json.

Simulator State (Internal Data Structure)

Used by the simulation service to track progress of each booking.

SimulationState:

booking_id: STRING

phase: STRING 'TO_PICKUP | TO_DROP | DONE'

current_step: INT

total_steps: INT

last_update_time: DATETIME

Application Deployment

(Parallel section to "Application Deployment" in the emergency doc.)

1. **Clone Repository**
    - Clone the GitHub repository containing backend and frontend code.
2. **Install Backend Dependencies**
    - Ensure Python 3 is installed.
    - Install required packages: Flask, flask-cors, python-dotenv, requests.
3. pip install -r requirements.txt
4. **Run the Flask Backend**
5. cd backend
6. python app.py
    - The server runs at a configured port (e.g., http://127.0.0.1:8000).
7. **Access Frontend Panels**
    - **User Panel** – for booking trucks and viewing assigned trip.
    - **Driver Panel** – for drivers to view bookings and accept them.
    - **Map View / Tracking Page** – to visualize real-time truck movement.

8. **Data Persistence**
    - All vehicles, locations, and booking updates are stored in backend/database/data.json.
    - This file acts as a lightweight database for demonstration and testing.

Sample API Payloads (Analogue of "Sample Query / Sample Data")

Sample User Booking Request

POST /api/user/book

Content-Type: application/json

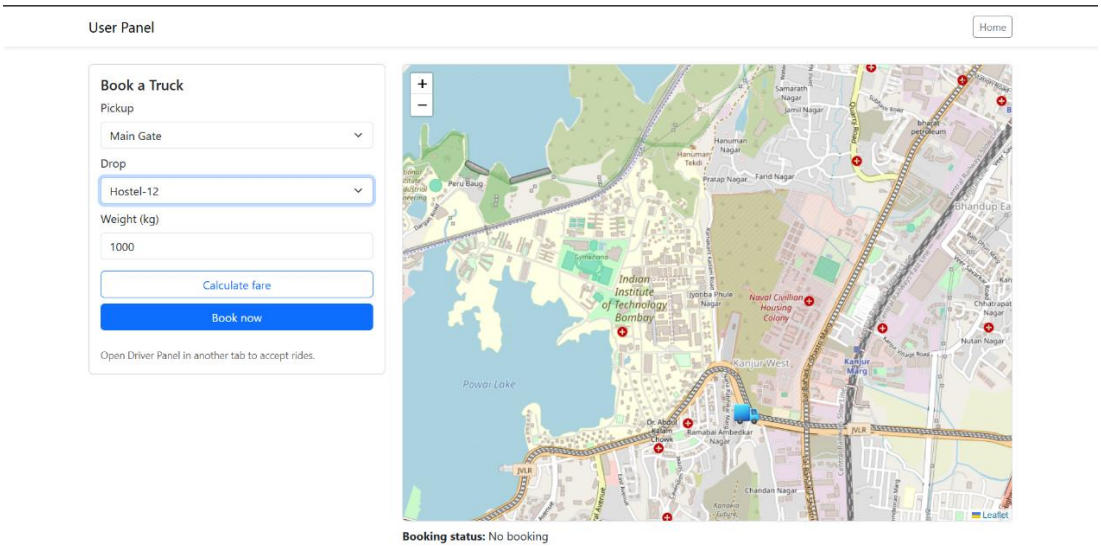

```
{
  "pickup": { "lat": 19.1334, "lon": 72.9133, "label": "IITB Main Gate" },
  "drop": { "lat": 19.1180, "lon": 72.9090, "label": "Logistics Hub" },
  "cargo_weight_kg": 5000
}
```


Sample Tracking Response

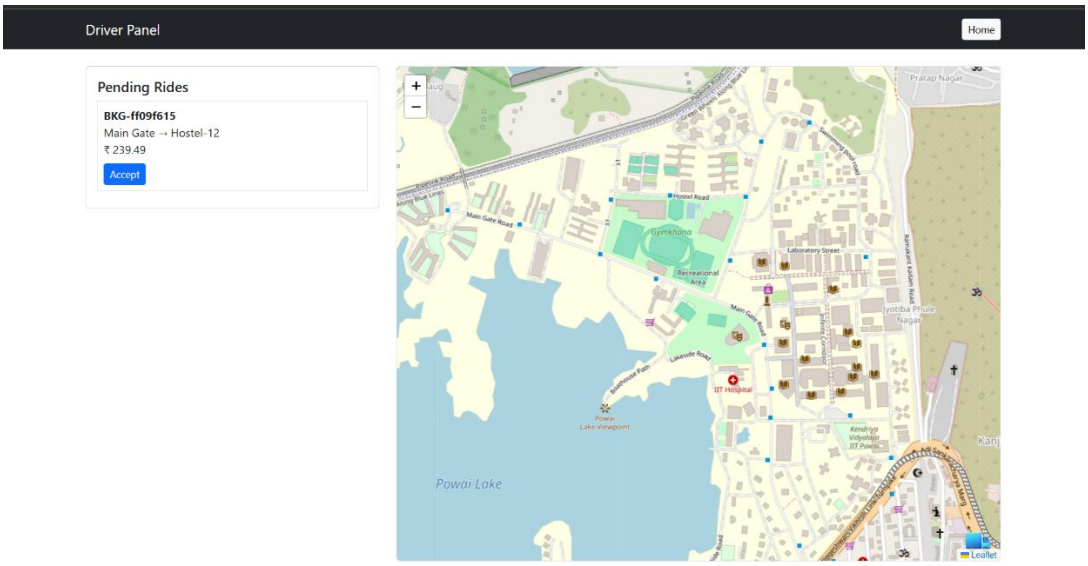

```
{
  "booking": {
    "id": "BOOK-123",
    "status": "ON_ROUTE_TO_DROP",
    "pickup": { "lat": 19.1334, "lon": 72.9133 },
    "drop": { "lat": 19.1180, "lon": 72.9090 }
  },
  "truck": {
    "id": "TRUCK-001",
    "current_location": { "lat": 19.1260, "lon": 72.9112 },
    "status": "ON_TRIP"
  }
}
```

Snapshots

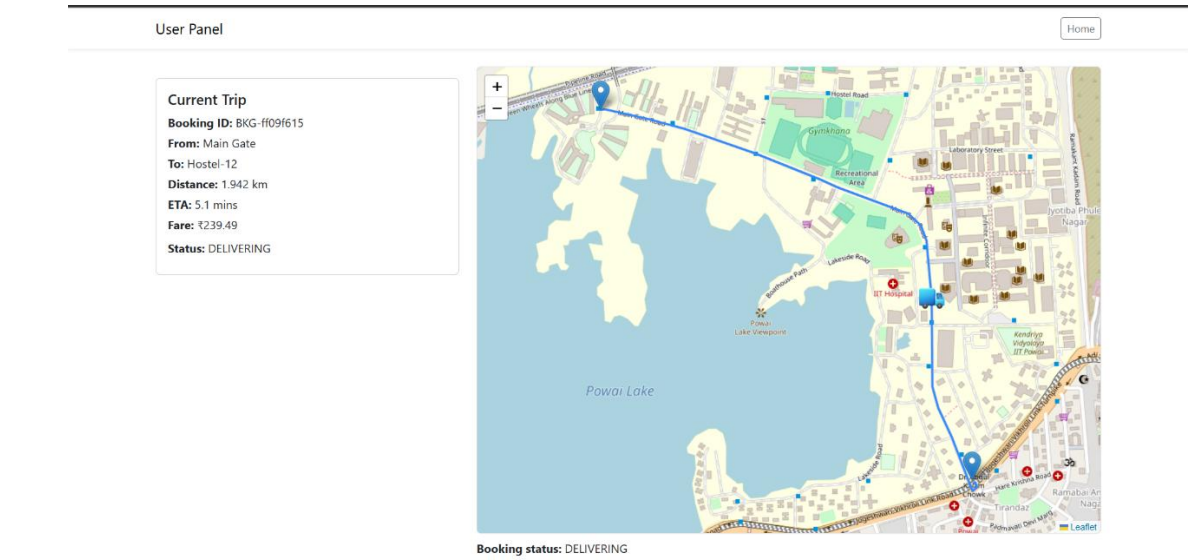
- **Figure 1: Truck Booking User Panel**
  – Screenshot showing map, pickup/drop selection, and booking form.

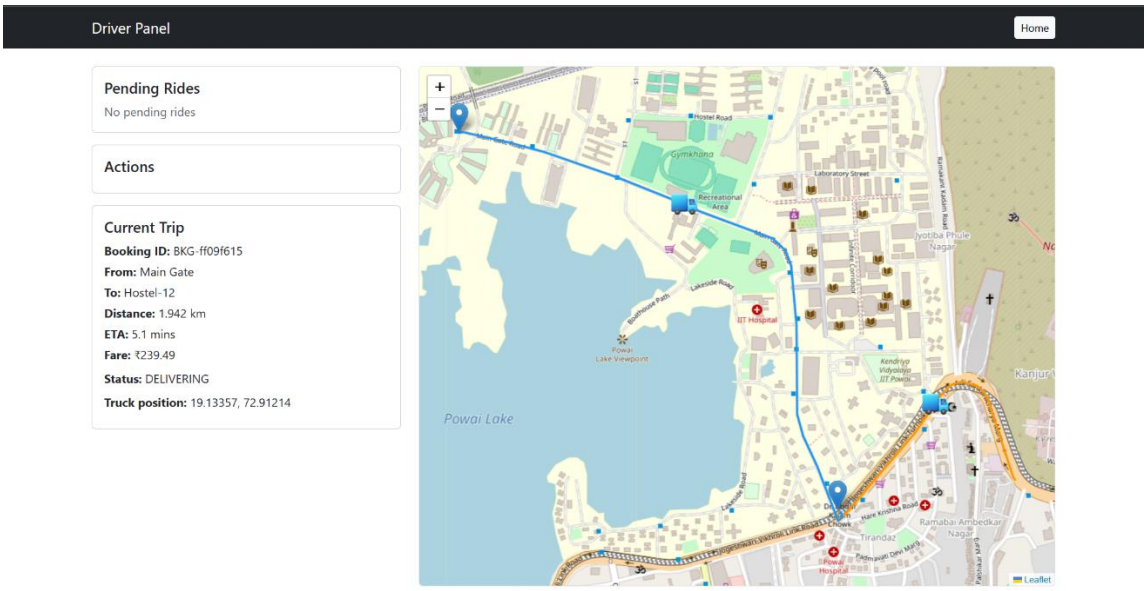


- **Figure 2: Driver Panel – Pending & Accepted Bookings**
  – Screenshot showing booking list with Accept buttons.



- **Figure 3: Trip Tracking View**
  – Screenshot of truck moving along the route with status.





User Panel

Home

Book a Truck

Pickup

Main Gate

Drop

Hostel-12

Weight (kg)

1000

Distance: 1.942 km | ETA: 5.1 mins | Cost: ₹239.49

Calculate fare

Book now

Trip delivered successfully.

Open Driver Panel in another tab to accept rides.

Booking status: COMPLETED